Cost controls — spend caps, monitoring & incident response

The consolidated cost-control runbook for a deployed Claude apps gateway: how spend enforcement works, how to set and change caps, how to watch spend, what a capped developer experiences, and what to do when the fail-closed spend store takes the fleet down. This document supersedes the spend-caps section of `om-runbooks.md` as the authoritative cost doc; alarm response in general stays in that document's §9, secret handling in §7.

Every command uses the repo's scripts and `deploy.env` variables — never hardcoded org values. Run operator commands from a host with `deploy.env` filled in and AWS credentials for the deployment region (scripts source `scripts/common.sh`, which loads `deploy.env`).

Verification status: the spend-cap admin API, both cap scopes, and read/write key separation were verified end to end against the **mirrored gateway binary + a throwaway Postgres** (2026-07-24), and client usage metrics are **DEPLOY-CONFIRMED flowing into AMP**. Everything else here — enforcement behavior on the deployed stack, the portal admin page (stack 04 is code-complete but not deploy-verified at all), the 429 experience on a real laptop, and the fail-closed outage drill — is **[NEEDS TEST-RUN CONFIRMATION]**. Individual sections repeat the tag where it matters.

---

1. Overview — how spend enforcement works

The master switch is configuration; the caps are data.

- **Master switch:** stack `02-gateway.yaml` renders the gateway's `admin:` block. The gateway runs spend enforcement **only** when `admin:` is configured — without it, even `enforcement.fail_closed_on_error` is inert. The stack also mints two API keys into Secrets Manager (`${NAME_PREFIX}/spend-admin-write-key`, `${NAME_PREFIX}/spend-admin-read-key`; `GenerateSecretString`, CMK-encrypted) and sets the retention knobs (`audit_retention_days: 365`, `spend_retention_months: 13`, `identity_retention_days: 90`).
- **Caps are DATA, not CloudFormation:** rows in the gateway's `spend_limits` table, written through `POST /v1/organizations/spend_limits`. **No cap rows = no enforcement**, so the stack is safe to deploy before any budget decisions exist, and cap changes never require a stack update.
- **Two metering paths, two authorities:**
- *Enforcement path:* client → gateway → **Postgres**. The gateway meters usage into the `spend` table as **aggregate cents per principal per period** (derived from tokens via the model rate table). This is what caps are checked against. Postgres does **not** hold per-request token counts.
- *Analytics path:* client → localhost ADOT **sidecar** → **AMP**. The `claude_code_*` metrics (`claude_code_cost_usage`, `claude_code_token_usage`, …) carry the per-request token/cost breakdown with `team` / `cost_center` / `user_groups` / `session_id` labels. This is what Grafana reads. AMP is authoritative for analytics only — an AMP outage never affects enforcement, and vice versa.
- **Scopes and precedence:** caps exist per **user** (`sub` or email), per **Okta group** (`rbac_group`), and **org-wide**. A per-user cap wins over group caps. When a user matches several group caps they combine per `SPEND_GROUP_LIMIT_MODE` (`min`, the default, takes the most restrictive — adding someone to a group can only tighten their cap; `max` takes the most permissive). Currency is USD only, enforced by the gateway.

- **Okta prerequisite for group caps:** per-group caps resolve against the **Okta groups claim**, which must actually be present in the token (the `groups` scope is requested unconditionally since 2026-07-24, but the claim itself is an org-side Okta app setting — see `../requests/okta-request-email.md`). Per-user and org-wide caps key on `sub` and keep working if the claim is missing; group caps then **silently match nothing**. Check `principal_emails.groups` via `scripts/diagnostics/dump-usage.sh` (§3.3) if group caps appear to have no effect.

---

2. Setting and changing caps

Trigger / Frequency: onboarding a team or user, a budget change, or a spend alert. Status: **[NEEDS TEST-RUN CONFIRMATION]** on the deployed stack (binary-verified 2026-07-24 against the mirrored gateway).

2.1 Preferred path — the portal admin page (individual identity)

With the download portal deployed (stack 04) and `PORTAL_ADMIN_GROUP` (04) plus `SPEND_ADMIN_GROUPS` (02) set to the same Okta group, members manage caps at `https://<GATEWAY_FQDN>/portal/admin` **as themselves**: the page walks the gateway's device-flow sign-in once per session, and every list/set/clear call rides the admin's own gateway token. The gateway re-checks group membership on each call and `admin_audit` records the individual actor (`oidc:<sub>`) — **no admin key is stored anywhere in the portal**.

Drift symptom: the page reports *"The gateway refused: your account is not in its spend-admin groups"* → `SPEND_ADMIN_GROUPS` and `PORTAL_ADMIN_GROUP` have diverged; re-align them and re-run the affected deploy script. An empty `SPEND_ADMIN_GROUPS` disables bearer-token admin entirely (only the generated keys work); an empty `PORTAL_ADMIN_GROUP` hides the admin page.

2.2 Break-glass path — `scripts/set-spend-limit.sh` (shared keys)

Works with nothing but `deploy.env` (`GATEWAY_FQDN`, `NAME_PREFIX`) and IAM access to the two key secrets — no portal, no Okta session:

```
scripts/set-spend-limit.sh --scope user       --id <okta-sub-or-email> --amount 50
scripts/set-spend-limit.sh --scope rbac_group --id <okta-group-name>   --amount 2500
scripts/set-spend-limit.sh --scope organization                        --amount 10000
scripts/set-spend-limit.sh --scope user --id <okta-sub-or-email> --clear   # remove a cap
scripts/set-spend-limit.sh --list                                          # review all caps
```

Mechanics that matter:

- `--amount` is **dollars** (`50` or `50.00`); the API takes whole **cents as a string** and the script converts exactly — no float rounding. `--period` is `daily` | `weekly` | `monthly` (default `monthly`).

- **Key hygiene:** the script pulls `${NAME_PREFIX}/spend-admin-write-key` (mutations) or `${NAME_PREFIX}/spend-admin-read-key` (`--list`) from Secrets Manager and hands the value to curl via a mode-600 **header file** — the key never appears on a command line (`ps` / `/proc` leak). Keep that property in any ad-hoc curl you write; better, don't write one.
- **TLS:** verification is against the system store **plus** `GATEWAY_CA_BUNDLE` and `EXTRA_CA_CERT_PATH` combined, so the script works both on a direct path (internal-PKI ALB cert) and behind TLS inspection. Never `-k`; on persistent failure the script prints the exact `openssl` command to compare the presented issuer against your bundle.
- Changes are **data-effective**: no stack update, image build, or service roll is needed for a cap to change. Always finish with `--list` to confirm what the gateway now holds.

---

3. Monitoring spend

3.1 Grafana — "Claude Code — Usage & Cost" dashboard

Provisioned from `docker/grafana/dashboards/claude-code-usage.json`; all panels honor the `Team`, `Cost center`, and `Okta group` variables (populated from the `team` / `cost_center` / `user_groups` metric labels).

Panel	What it answers
Cost / **Tokens** / **Sessions** / **Active users** (stat row)	Totals for the selected range; sessions counted by distinct `session_id`, active users (24h) by distinct `user_email`
Cost by team / **by cost center** / **by Okta group**	Hourly spend split along each org dimension — your first stop for "who is driving cost"
Tokens by model	Model mix (e.g. Opus vs Sonnet) — a cheap lever when cost spikes
Tokens by type (input / output / cache)	Cache effectiveness and prompt-heavy workloads
Top users by cost (selected range)	`topk(15)` table by `user_email` with team/cost-center — the candidates for a per-user cap
Lines of code changed / **Commits created**	Output-side context so cost is read against delivered work

Query note: the panels compute per-window deltas via **window functions** (`max_over_time - min_over_time`), not `increase()` — reworked after the `session.id` label fix (§6). Empty `Okta group` dropdown → the groups claim is not landing; see §1's prerequisite and §3.3.

3.2 Direct AMP queries — `scripts/diagnostics/amp-query.py`

SigV4-signed report against the AMP workspace (uses botocore's full credential chain). Env-driven: `OBSERVABILITY_AMP_ENDPOINT` (persisted into `deploy.env` by stack 03), `AWS_REGION`, and `AMP_QUERY_WINDOW_HOURS` (default 48 — client metrics are bursty; short windows miss them).

```
. scripts/deploy.env
python3 scripts/diagnostics/amp-query.py
```

It reports which `claude_code_*` metric names are stored, the `otelcol_*` heartbeat series, and — when client metrics are missing — walks the collector's own pipeline counters to a verdict (accepted vs refused vs **failed translations**, the silent-drop counter). Its 403 hints are load-bearing: `SignatureDoesNotMatch` is an encoding regression, not IAM; a plain 403 on a CMK-encrypted workspace usually means the **caller** lacks `kms:Decrypt` (`kms:ViaService=aps.<region>.amazonaws.com`) — the exact trap that broke Grafana on 2026-07-23.

3.3 Postgres ground truth — `scripts/diagnostics/dump-usage.sh`

Read-only dump of what the gateway has actually persisted, over the same connection path the gateway uses (`${NAME_PREFIX}/db-app-user` secret + RDS CA, verify-full). Run from an in-VPC host or bastion whose ENI carries the DB client SG (stack 01 output `DBClientSecurityGroupId`) and with IAM to read the app-user secret; `pg8000` installs offline from `docker/db-admin/vendor`. `DUMP_LIMIT` caps rows per table (default 50).

Table	Contents	Retention
`spend`	aggregate cents per principal per period — the enforcement ledger	13 months
`spend_limits`	the configured caps (what `--list` shows)	live data
`principal_emails`	principal → email/name + the resolved **Okta groups claim**	90 days
`admin_audit`	every admin API mutation, with actor attribution	365 days

Interpretation: empty `spend` → the gateway is metering nothing (no inference has flowed, or metering is broken); NULL/empty `principal_emails.groups` → group caps match nothing and the Grafana Okta group filter is empty. **Raw per-request token/cost detail lives only in AMP** — Postgres holds aggregates; use §3.1/§3.2 for the breakdown and `scripts/diagnostics/diagnose-telemetry.sh` for pipeline health.

---

4. What a capped user experiences — and lifting a cap fast

A developer over their cap gets **HTTP 429**, error type `billing_error`, message *"spend limit reached — "*, with `x-should-retry: false`. It is **not** a transient error: the client will not retry around it, and waiting does not help until the period rolls over or the cap changes. Set `SPEND_BLOCKED_MESSAGE` in `deploy.env` to org- specific routing text ("contact for an increase") — it is the only self-service breadcrumb the developer sees. Status: **[NEEDS TEST-RUN CONFIRMATION]** on a real client.

Confirm it is a cap (60 seconds):

```
scripts/set-spend-limit.sh --list        # is there a cap matching this user / their groups / org?
scripts/diagnostics/dump-usage.sh        # spend row for the principal vs the cap amount
```

One user capped with caps listed and a matching `spend` row → working as designed. **Everyone** capped at once → this is probably not a cap at all; go straight to §5.

Lift or raise:

```
scripts/set-spend-limit.sh --scope user --id <okta-sub-or-email> --amount <new-dollars>
# or remove entirely:
scripts/set-spend-limit.sh --scope user --id <okta-sub-or-email> --clear
```

Data-effective immediately on the gateway side (no redeploy); have the user retry, and remember `min` mode means a generous per-group cap cannot loosen a tighter one — a per-user cap is the reliable override, since per-user always wins.

---

5. INCIDENT RUNBOOK — fail-closed spend-store outage

Trigger / Frequency: fleet-wide 429s reported by developers, or DB alarms. **No dedicated CloudWatch alarm watches this condition** — detection today is user reports plus the DB-side alarms in `om-runbooks.md` §9. Status: **[NEEDS TEST-RUN CONFIRMATION]** — the posture is a deliberate template setting; the outage path has never been exercised.

Why this exists: `enforcement.fail_closed_on_error: true` is a hardcoded operator decision (2026-07-24) in `cloudformation/02-gateway.yaml`'s rendered gateway config. If the spend store is unreachable or errors, the gateway returns 429 **for every request** rather than allow uncapped spend. This is a deliberate availability trade: **an RDS/spend-store outage halts all inference fleet-wide, not just cost tracking.**

Symptoms:

- Sudden fleet-wide 429s ("spend limit reached" / "spend limit unavailable") affecting **every** user simultaneously — including users with no cap set.
- `spend check failed` / `store_error` entries in the gateway log group.
- Correlated RDS trouble: `${NAME_PREFIX}-db-rotation-errors` alarm, RDS instance events, or a recent DB credential rotation.

Triage (in order):

1. Confirm scope: one user capped = §4, not an incident. All users = here.
2. Check the gateway log group for `spend check failed` / `store_error`.
3. Check RDS: instance status/events in the console, storage/connection exhaustion, and the `${NAME_PREFIX}-db-rotation-errors` alarm — a botched app-credential rotation looks exactly like a store outage to the gateway (recovery in `om-runbooks.md` §3).

4. Check network path: DB SG membership unchanged, gateway tasks healthy.

Recovery:

1. **Fix the database** — that is the real fix; enforcement recovers on its own once the store answers.
2. **Break-glass (only if inference must resume before the store is fixed):** flip `fail_closed_on_error: true` → `false` in the `enforcement:` block of `cloudformation/02-gateway.yaml` (it is a template literal, not a parameter) and re-run `scripts/deploy-gateway.sh`. Spend is now **unmetered and uncapped** while the store is down.
3. **Obligation:** the flip is temporary by definition. Track it as an open incident action; once the store is healthy, revert the template edit, re-run `scripts/deploy-gateway.sh`, and confirm enforcement is back with `scripts/set-spend-limit.sh --list` plus a capped-user probe. Do not commit the flipped value — the repo posture is fail-closed.

Verification after recovery: `--list` succeeds; `spend` rows advance again (`scripts/diagnostics/dump-usage.sh`); no `store_error` in fresh gateway logs.

---

6. Gaps & recommendations (honest)

- **No AWS-account-level budget alarm exists.** Nothing in `cloudformation/` creates an `AWS::Budgets::Budget` or Cost Explorer–based alarm; the alarms that do exist (§9 of `om-runbooks.md`, `MissingTelemetryAlarm` in `cloudformation/03-observability.yaml`) watch **pipeline health**, not dollars. Today's defenses are app-level caps only — which enforce nothing until someone writes cap rows, and which a fail-closed flip (§5.2) temporarily disables. **Recommendation:** add an AWS Budgets alert (or Cost Explorer anomaly detection) on this account's **Bedrock** spend as an independent backstop that catches runaway cost even when app-level enforcement is off, misconfigured, or bypassed. Deliberately left as a recommendation — budgets are org policy, and GovCloud billing-data flows vary by agreement; decide placement with the finance owner before adding CFN.
- **The org-wide cap is the closest existing backstop.** Until a budget alarm exists, consider a generous `--scope organization` cap sized well above expected monthly spend, so a metering-side runaway hits *something*.
- `session.id` **cardinality fix — context for spend numbers.** The sidecar previously deleted `session.id` for cardinality; concurrent sessions from one user then interleaved onto a single series as a sawtooth and `increase()` **drastically inflated** dashboard spend (observed live). `session.id` is now kept — each session is its own monotonic series — and the dashboards were reworked to window functions accordingly. Committed; confirm the deployed sidecar carries it after the next `deploy-gateway.sh` run **[NEEDS TEST-RUN CONFIRMATION]**. Treat pre-fix dashboard history as unreliable; Postgres `spend` was and remains the enforcement ledger.
- **Not yet live-verified** (mirrored-gateway/throwaway-Postgres verified only): cap enforcement and the 429 experience on the deployed stack, the portal admin page end to end (stack 04 has no deploy verification at all), bearer-token admin via `SPEND_ADMIN_GROUPS`, the fail-closed outage and break-glass drill, and admin-key rotation. All **[NEEDS TEST-RUN CONFIRMATION]**.

---

7. Audit trail

Every spend-limit mutation lands in the gateway's `admin_audit` table (365-day retention), attributed to the acting identity:

- `oidc:<sub>` — an individual admin acting through the portal admin page (or any bearer-token call allowed by `SPEND_ADMIN_GROUPS`).
- **key** `id` (`deploy-write` / `deploy-read`) — the break-glass CLI's shared keys. This is why the portal path is preferred: shared-key entries identify a credential, not a person.

The portal admin page shows this trail read-only and additionally writes `event: portal_admin` lines (connects, actions, denials) to the portal's own audit log group. Inspect the table directly with `scripts/diagnostics/dump-usage.sh` (§3.3).

Key rotation: both admin keys are `GenerateSecretString` secrets, injected as ECS `Secrets` and read only at container start. Rotate by writing a new value with the file-based no-argv pattern and then **forcing a new deployment** (`aws ecs update-service --cluster <cluster> --service ${NAME_PREFIX}-gateway --force-new-deployment`) — the same procedure as the gateway JWT secret in `om-runbooks.md` §7. A plain `deploy-gateway.sh` re-run is NOT enough: the secret write happens outside CloudFormation, so the stack update is an empty changeset and running tasks keep the old keys. Rotating the write key invalidates any operator's cached copy but not portal admins, who never use it.